

EICAS Primary Parameters Display Simülasyonu

İbrahim Etem YILMAZ, Mehmet Akif ŞAKIR, Yasin AŞA, Yaşar Burak ÖZKAYA, Yusuf ŞAHİN

Havacılık Elektrik ve Elektronik Bölümü
Havacılık ve Uzay Bilimleri Fakültesi, Eskişehir Teknik Üniversitesi
HYO338C – Elektronik Gösterge Sistemleri
Öğr. Gör. Dr. Aziz KABA
18.05.2026, Eskişehir

Özet — Bu rapor, Boeing 737-600/700 uçaklarında kullanılan EICAS (Engine Indicating and Crew Alerting System) Primary Parameters Display göstergesinin web arayüzünde HTML, CSS ve JavaScript kullanılarak gerçeğe yakın bir biçimde simüle edilmesini detaylandırmaktadır.

İçindekiler

1 Giriş ve Projeye Genel Bakış	2	5.2.2 Eylemsizlik (Inertia) Sabiti	7
1.1 Projenin Amacı ve Kapsamı	2	5.2.3 Durum Değişkenleri	7
1.2 EICAS Sistemine Giriş	2	5.3 Motor Fizik Modeli	7
1.2.1 EICAS Sisteminin Görevi	2	5.3.1 Ana Motor Güncelleme Fonksiyonu	7
1.2.2 Simüle Edilen Parametreler	2	5.3.2 Eylemsizlik (Atalet) Simülasyonu	7
1.3 Proje Dosya Yapısı	2	5.4 İbre Açısı Hesaplamaları	8
2 SVG Gösterge Tasarımı	2	5.4.1 N1 İbre Açısı	8
2.1 Kaynak Materyalin İşlenmesi	2	5.4.2 EGT İbre Açısı	8
2.1.1 PDF'den SVG'ye Dönüşüm	2	5.4.3 N1 Referans (Bug) Açısı	8
2.1.2 Inkscape ile Manuel Yeniden Çizim	2	5.4.4 SVG Süpürme Yay (Sweep Arc) Hesabı	8
2.2 SVG Dosyası Yapısı	3	5.5 Limit Kontrolleri ve Renk Uyarıları	8
2.3 SVG'nin HTML'e Entegrasyonu (Inline SVG)	3	5.6 Yakıt Simülasyonu	8
2.3.1 Inline SVG Yönteminin Avantajları	3	5.6.1 Tank Yönetim Mantiği	8
2.4 SVG Öğeleri ve ID Yapısı	3	5.6.2 Yakıt Göstergesi Yay Hesabı	9
2.4.1 İbre Dönme Mekanizması	3	5.7 Arıza ve Uyarı Sistemi	9
3 HTML Yapısı ve Sayfa İskeleti	4	5.7.1 EICAS Uyarı Mekanizması	9
3.1 Genel Sayfa Yapısı	4	5.7.2 Arıza Türleri ve Etkileri	9
3.2 Motor Kontrol Paneli Yapısı	4	5.7.3 Yağ Kaybı Fizik Etkisi	9
3.2.1 Gaz Kolu Kaydırıcıları	4	5.8 Ana Motor Döngüsü (requestAnimationFrame)	10
3.2.2 Arıza Düğmeleri	4	5.9 Uçuş Senaryoları	10
3.3 Referans ve Sıcaklık Kontrolleri	4	5.9.1 Zamanlayıcı Altyapısı	10
3.3.1 Birim Dönüşüm Aracı (Toggle Widget)	4	5.9.2 Normal Kalkış Senaryosu	10
3.4 Uçuş Modu Paneli	4	5.9.3 Motor Arızası (V1 Cut) Senaryosu	10
3.5 Uçuş Modları Referans Tablosu	5	6 Matematiksel Formüller ve Fizik Modeli	10
4 CSS ile Görsel Tasarım ve Animasyonlar	5	6.1 Görselleştirme ve Açısı Hesaplamaları	10
4.1 Tasarım Dili ve Renk Paleti	5	6.2 Temel Motor Parametreleri	11
4.2 Düzen ve Ölçeklendirme	5	6.3 Eylemsizlik (Inertia) Simülasyonu	11
4.2.1 Genel Sayfa Düzeni	5	6.4 Arıza ve Sistem Etkileşimleri	11
4.2.2 EICAS Ekranının Kare Oranı	5	6.5 Tüketim ve Birim Çevrimleri	11
4.2.3 Zoom ile Ölçeklendirme	6	7 Kullanılan Araçlar ve Geliştirme Ortamı	11
4.3 EICAS Uyarı Animasyonları	6	7.1 Kullanılan Diller ve Araçlar	11
4.4 Senaryo Aktif Parlama Etkisi	6	7.2 JavaScript Kullanımı	12
4.5 Birim Dönüşüm Aracının CSS Animasyonu	6	7.3 SVG Kullanımı	12
4.6 Duyarlı Tasarım	6	7.4 Tarayıcı Uyumluluğu	12
5 JavaScript ile Motor Fiziği Simülasyonu	7	7.5 Yayınlama ve Kurulum	12
5.1 Mimari Genel Bakış	7	8 AI Kullanımı	12
5.2 Sabitler ve Durum Yönetimi	7	8.1 EICAS Ekranının Çizimi ve SVG İşlemleri	12
5.2.1 Rotasyon Sabitleri	7	8.2 Kodlama	12
		8.3 Raporlama	13
		9 Sonuç ve Değerlendirme	13
		9.1 Genel Değerlendirme	13
		9.2 Proje Sürecindeki Kazanımlar	13

1. Giriş ve Projeye Genel Bakış

1.1. Projenin Amacı ve Kapsamı

Bu projenin temel hedefi, Boeing 737-600/700 uçaklarında kullanılan **EICAS (Engine Indicating and Crew Alerting System) Primary Parameters Display** göstergesini web arayüzünde gerçeğe yakın bir biçimde simüle etmektir. Proje; HTML, CSS ve JavaScript kullanılarak oluşturulan tamamen statik, sunucu gerektirmeyen bir web uygulaması olarak tasarlanmıştır. Bu tercih, uygulamanın herhangi bir tarayıcıda doğrudan çalıştırılabilmesine olanak tanımakta ve kullanıcıya bağımlılık gerektirmeyen, taşınabilir bir deneyim sunmaktadır.

Canlı Uygulama

Uygulama https://acrilot.github.io/EICAS_Uygulama/ adresi üzerinden doğrudan erişilebilir durumdadır.

1.2. EICAS Sistemine Giriş

EICAS (Engine Indicating and Crew Alerting System), modern Boeing uçaklarında motor parametrelerini izlemek ve mürettebatı kritik arızalar konusunda uyarmak amacıyla tasarlanmış entegre bir aviyonik sistemidir. Bu sistem, uçuş güvertesindeki çok işlevli ekranlar aracılığıyla pilotlara sürekli veri akışı sağlar.

1.2.1. EICAS Sisteminin Görevi

EICAS Primary Parameters Display’de görüntülenen başlıca veriler şunlardır:

- Gerçek zamanlı motor parametreleri (N1, N2, EGT, yağ basıncı, yağ sıcaklığı, yakıt akışı, titreşim)
- Görsel ve metin tabanlı uyarılarla sistem arıza durumları
- Aktif uçuş modu ve itki limiti durumu
- Kanat ve merkez tanklar için yakıt durumu

1.2.2. Simüle Edilen Parametreler

Bu projede simüle edilen başlıca motor ve sistem parametreleri aşağıdaki tabloda özetlenmiştir:

Tablo 1: Simüle Edilen Motor Parametreleri

Parametre	Kısaltma	Birimi
Fan Hızı (1. Şaft)	N1	%
Kompresör Hızı (2. Şaft)	N2	%
Egzoz Gazı Sıcaklığı	EGT	°C
Yağ Basıncı	OIL PRESS	PSI
Yağ Sıcaklığı	OIL TEMP	°C
Yağ Miktarı	OIL QTY	Quart
Yakıt Akışı	FF	PPH
Titreşim	VIB	B. Yok
Top. Hava Sıc.	TAT	°C
Kanat Yakıtı	FUEL	LB/KG
Merkez Tank	CTR FUEL	LB/KG

1.3. Proje Dosya Yapısı

Uygulama üç ana dosya üzerine inşa edilmiştir:

- `index.html` - Inline SVG ve arayüzün temel iskeleti
- `style.css` - Arayüz görsel tasarımı, animasyonları ve düzeni
- `script.js` - Motor fiziği simülasyonu, event listener’lar ve senaryo sistemi

Ayrıca projenin `Fonts/` dizininde bulunan `OCRBPPro.TTF` fontu, orijinal EICAS ekranının OCR-B tabanlı karakter setini yansıtmak için kullanılmaktadır.

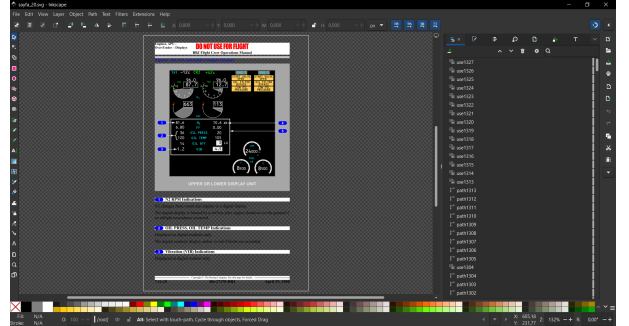
2. SVG Gösterge Tasarımı

2.1. Kaynak Materyalin İşlenmesi

Simülasyonun görsel çekirdeğini oluşturan EICAS göstergesi, Boeing 737’nin resmi *Flight Crew Operations Manual (FCOM)* [1] belgesinden türetilmiştir. Bu süreçte belgenin *Engines, APU* bölümünün *Over/Under Displays* kısmında yer alan teknik gör-seller, aşağıdaki işlemlerden geçirilmiştir:

2.1.1. PDF’den SVG’ye Dönüştürme

İlk olarak FCOM’un ilgili sayfaları bir Python scripti kullanılarak SVG formatına dönüştürülmüştür. Bu aşamada elde edilen ham SVG dosyası, ilgili sayfadaki tüm yazıların ve teknik çizimlerin bir bütünü halindedir. Ancak bu ham çıktı doğrudan kullanılacak nitelikte değildir; zira bu SVG içerisinde yer alan teknik çizimler onlarca birbirinden bağımsız yol (*path*) ve düğüm (*node*) içermekte, anlamlı gruplamalar ya da ID atamaları içermemektedir.

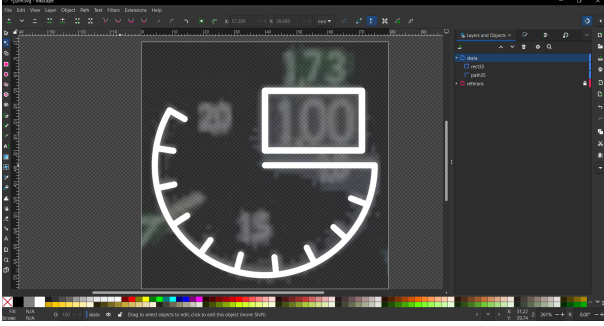


2.1.2. Inkscape ile Manuel Yeniden Çizim

Bu sorunun üstesinden gelmek için açık kaynaklı vektör çizim uygulaması Inkscape kullanılmıştır. İşlem adımları şu şekilde özetlenebilir:

- Temizleme:** Ham SVG’deki ok işaretleri, sayfa numaraları, açıklama metinleri ve gereksiz dekoratif öğeler kaldırılmıştır.
- Maske katmanı oluşturma:** Temizlenmiş çizimler, altına bir referans kılavuzu olarak işlev görecektir şeffaf bir *maske katmanı* haline getirilmiştir.
- Temiz katman üzerine yeniden çizim:** Maskenin üstüne boş bir yeni katman eklenmiş ve tüm gösterge öğeleri bu katman üzerinde, maske göz önünde bulundurularak sıfırdan ve tek parça olarak elle yeniden çizilmiştir.

4. **ID ataması:** JavaScript kodu tarafından dinamik olarak değiştirilecek tüm SVG öğelerine (ibreler, metin alanları, uyarı kutuları, yakıt barları, mod göstergeleri) anlamlı ve benzersiz id nitelikleri atanmıştır.



Neden Yeniden Çizim?

PDF'den elde edilen SVG dosyalarında her gösterge öğesi, aralarında anlamlı ilişki bulunmayan onlarca bağımsız *path* segmentinden oluşmaktadır. Bir ibrenin tek parça bir `<path>` öğesi olması gerekirken, ham çıktıda aynı ibre onlarca parçaya bölünmüş olabilmektedir. Bu durum hem ID atamayı hem de JavaScript ile dönüşüm uygulamayı olanaksız kılmaktadır. Yeniden çizim, bu problemi tamamen ortadan kaldırmaktadır.

2.2. SVG Dosyası Yapısı

Tamamlanan çizim iki ayrı formatta saklanmıştır:

- **INKSCAPE_UL.svg:** Inkscape'e özgü XML ad alanları, katman yapısı, kılavuz çizimleri ve düzenleyici meta verilerini barındıran tam sürüm. Bu dosya, düzenleme ve gelecekteki revizyonlar için referans olarak saklanmıştır.
- **PLAIN_UL.svg:** Yalnızca standart SVG öğelerini içeren, düzenleyiciye özgü tüm meta verilerden arındırılmış sade sürüm. Kod tabanında Inline SVG olarak gömülmüş olan bu dosyadır; daha kısa ve verimlidir.

2.3. SVG'nin HTML'e Entegrasyonu (Inline SVG)

SVG dosyasının HTML içine gömülmesi için **Inline SVG** yöntemi seçilmiştir. Bu yöntemde, SVG dosyasının tüm XML içeriği doğrudan `index.html` dosyasının ilgili `<div>` etiketi içine yerleştirilmektedir:

```
1 <div class="eicas-display">
2   <svg viewBox="0 0 67.436997 67.436997"
3     version="1.1" id="svg1" xmlns="http://www.
4       w3.org/2000/svg"
5     xmlns:svg="http://www.w3.org/2000/svg">
6     <defs id="defs1" />
7   </svg>
8 </div>
```

Kod Parçası 1: Inline SVG Entegrasyonu

2.3.1. Inline SVG Yönteminin Avantajları

Inline SVG yönteminin harici SVG dosyası yöntemine kıyasla sunduğu avantajlar şunlardır:

- **Doğrudan DOM erişimi:** JavaScript, `document.getElementById()` ile SVG öğelerine doğrudan erişebilmektedir.
- **CSS entegrasyonu:** SVG öğeleri, aynı HTML öğeleri gibi `style.css` dosyasındaki kurallardan etkilenebilmektedir.
- **Animasyon desteği:** CSS `@keyframes` animasyonları ve JavaScript tarafından uygulanan `transform` stilleri, SVG öğelerine kısıtlama olmaksızın uygulanır.
- **Sunucu bağımsızlığı:** Ek bir HTTP isteğine gerek kalmaksızın çalışır; yerel dosya sisteminden (`file://`) açılabilir.

2.4. SVG Öğeleri ve ID Yapısı

Projede kullanılan SVG öğeleri ve bunlara atanan ID'ler belirli bir isimlendirme kuralına göre düzenlenmiştir:

Tablo 2: Temel SVG Öğeleri Kategorileri

Kategori	Örnek ID	İşlev
N1 İbreleri	n1_needle_l	Açısı değişir
EGT İbreleri	egt_left_needle	Açısı değişir
Ref. Gösterge	n1_ref_bug_l	Referans açısı
Değer Metni	val_n1_l	Metin içeriği
Yakıt Yayları	fuel_arc_1	dashoffset
Süpürme Yayı	n1_left_sweep	dashoffset
Uyarı Kutusu	lop_bg	Görünürlük
Uyarı Metni	lop_text	CSS sınıfı
Uçuş Modları	mode_to	Görünürlük

2.4.1. İbre Dönme Mekanizması

SVG ibreleri, CSS `transform: rotate()` özelliği kullanılarak döndürülmektedir. Doğru dönme merkezini sağlamak için iki kritik CSS kuralı uygulanmıştır:

```
1 #n1_needle_l, #n1_needle_r,
2 #egt_left_needle, #egt_right_needle,
3 #n1_ref_bug_l, #n1_ref_bug_r {
4   transform-box: fill-box; /* 0ge sınır
5     kutusu */
6   transform-origin: 0% 50%; /* Pivot noktasi
7     */
8 }
```

Kod Parçası 2: İbre Dönme Merkezi Ayarı

`transform-box: fill-box` özelliği, dönme merkezinin SVG koordinat sistemine değil, öğenin kendi sınır kutusuna (*bounding box*) göre hesaplanmasını sağlar. `transform-origin: 0% 50%` ise dönmenin öğenin sol kenarının (0%, x-ekseni) tam ortasından (50%, y-ekseni) gerçekleştirilmesini belirtir. Bu nokta ibrenin pivot noktasıyla örtüşmektedir.

3. HTML Yapısı ve Sayfa İskeleti

3.1. Genel Sayfa Yapısı

index.html dosyası üç ana bölümden oluşmaktadır:

1. **<head> bölümü:** Karakter seti tanımı, CSS bağlantısı ve sayfa meta verileri
2. **EICAS Göstergesi (.eicas-display):** Inline SVG içeren görüntüleme alanı
3. **Kontrol Panelleri (.control-panel):** Motor kontrol paneli ve uçuş modu paneli

Genel sayfa iskeletinin basitleştirilmiş görünümü şu şekildedir:

```
1 <body>
2   <div class="eicas-display">
3     <svg></svg>
4   </div>
5   <div class="control-panel engine-panel">
6   </div>
7   <div class="control-panel mode-panel">
8   </div>
9 </body>
```

Kod Parçası 3: Genel Sayfa Yapısı

3.2. Motor Kontrol Paneli Yapısı

Motor kontrol paneli (.engine-panel), simetrik bir yerleşim düzenine sahiptir. Solda sol motora ait arıza düğmeleri, ortada her iki motoru kontrol eden dikey kaydırıcılar ve sağda sağ motora ait arıza düğmeleri bulunmaktadır.

3.2.1. Gaz Kolu Kaydırıcıları

Her motor için 0–105 aralığında değer alan dikey kaydırıcılar ve ilişkili metin girdi kutuları kullanılmaktadır:

```
1 <div class="slider-container">
2   <span class="input-value input-vertical">
3     <input type="number" id="throttle-val-L"
4       class="input-value"
5       min="0" max="105" step="0.1" value="
6       0" />
7   </span>
8   <input type="range" id="throttle-L"
9     class="slider-vertical" min="0" max="
10    105"
11    step="0.1" value="0" />
12 </div>
```

Kod Parçası 4: Motor Kaydırıcı Yapısı

3.2.2. Arıza Düğmeleri

Her motor için altı farklı arıza durumu, onclick nitelikleri aracılığıyla JavaScript fonksiyonlarına bağlanmış düğmelerle kontrol edilmektedir:

```
1 <button onclick="toggleReverser('L')">REV </
2   button>
3 <button onclick="toggleFailure('eng','L')">ENG
4   FAIL </button>
5 <button onclick="toggleFailure('svo','L')">SVO <
6   /button>
7 <button onclick="toggleFailure('ofb','L')">OFB <
8   /button>
9 <button onclick="toggleFailure('lop','L')">LOP <
10  /button>
```

```
6 <button onclick="toggleFailure('thrust','L')">
7   THRUST </button>
```

Kod Parçası 5: Arıza Düğmesi Örneği

3.3. Referans ve Sıcaklık Kontrolleri

N1 referans değeri için bir yatay kaydırıcı ve yanında metin girdi kutusu, seçilmiş sıcaklık değeri için ise kaydırıcı ve birim dönüşüm aracı bulunmaktadır.

3.3.1. Birim Dönüşüm Aracı (Toggle Widget)

Sıcaklık birimi ve yakıt birimi değiştirme için özel olarak tasarlanmış animasyonlu bir geçiş aracı kullanılmaktadır. Bu araç standart bir onay kutusu (checkbox) yerine tamamen CSS ve JavaScript ile inşa edilmiştir:

```
1 <div class="unit-toggle-widget" onclick="
2   toggleTemperatureUnit()">
3   <div class="unit-toggle-track">
4     <span class="unit-opt active" id="temp-
5       opt-c">C </span>
6     <span class="unit-opt" id="temp-
7       opt-f">F </span>
8   </div>
9   <div class="unit-toggle-slider" id="temp-
10    unit-slider"></div>
11 </div>
```

Kod Parçası 6: Toggle Widget HTML Yapısı

3.4. Uçuş Modu Paneli

Uçuş modu paneli (.mode-panel), 3 × 3 ızgara düzeninde dokuz uçuş modu düğmesi, bir temizleme düğmesi ve uçuş senaryosu kontrolleri içermektedir:

```
1 <div class="flight-modes-grid">
2   <button onclick="setFlightMode('mode_r_to')">
3     R-T0 </button>
4   <button onclick="setFlightMode('mode_r_crz')">
5     R-CRZ </button>
6   <button onclick="setFlightMode('mode_to')">
7     T0 </button>
8   <button onclick="setFlightMode('mode_clb')">
9     CLB </button>
10  <button onclick="setFlightMode('mode_crz')">
11    CRZ </button>
12  <button onclick="setFlightMode('mode_ga')">
13    G/A </button>
14  <button onclick="setFlightMode('mode_con')">
15    CON </button>
16  <button onclick="setFlightMode('mode_---')">
17    - - - </button>
18  <button onclick="setFlightMode('mode_at_lim')">
19    A/T LIM </button>
20 </div>
21 <button class="clear-mode-btn" onclick="
22   setFlightMode('NONE')">
23   MODU TEMİZLE
24 </button>
```

Kod Parçası 7: Uçuş Modu Izgarası

3.5. Uçuş Modları Referans Tablosu

Tablo 3: Uçuş Modları Referansı

Mod	Açılımı	İşlevi
R-TO	Reduced Take-Off	FMC üzerinden girilen varsayılan sıcaklık değeriyle hesaplanmış düşürülmüş kalkış itki limitinin aktif olduğunu belirtir.
R-CRZ	Reduced Cruise	Seyir aşamasında motor ömrünü korumak amacıyla uygulanan kısıtlı seyir itki limitinin devrede olduğunu ifade eder.
TO	Take-Off	Kalkış için hesaplanan standart maksimum itki limitinin kullanımda olduğunu gösterir.
CRZ	Cruise	Seyir safhası için belirlenen standart itki limitinin aktifleştğini belirtir.
CLB	Climb	Tırmanma safhası için hesaplanan itki limitinin devrede olduğunu gösterir.
G/A	Go-Around	İnişten vazgeçilmesi durumundaki acil maksimum itki limitinin aktif olduğunu ifade eder.
CON	Continuous	Bir motor arızasında diğer motorla hasar görmeden sürekli uçulabilecek maksimum itki limitini belirtir.
- - -	—	Aktif bir itki limit modunun devrede olmadığını gösterir.
A/T LIM	Autothrottle Limit	Otomatik Gaz sisteminin motorları korumak için devreye girdiğini ve devri sınırlan- dığını belirtir.

4. CSS ile Görsel Tasarım ve Animasyonlar

4.1. Tasarım Dili ve Renk Paleti

Kontrol panelinin görünümü, EICAS ekranının görünümü ile uyumlu olacak şekilde tasarlanmıştır. `style.css` dosyası, bu estetiği yansıtmak için CSS değişkenleri üzerine inşa edilmiştir [2]:

```
1 :root {
2   --panel-bg: #161c22; /* Panel arka
   plan rengi */
3   --panel-border: #252e38; /* Panel çerçeve
   rengi */
4   --section-border: rgba(0, 229, 255, 0.08);
   /* Bölüm ayracı */
5   --accent: #00e5ff; /* Ana vurgu
   rengi (cyan) */
6   --accent-dim: rgba(0, 229, 255, 0.5);
7   --text: #c8d6e0; /* Ana metin
   rengi */
8   --text-dim: #6b7f8f; /* İkincil metin
   rengi */
9   --btn-bg: #1c252f; /* Buton arka
   plan rengi */
10  --danger: #ff4444; /* Tehlike rengi
   (kirmizi) */
11  --warn: #ffb300; /* Uyarı rengi (
   amber) */
12 }
```

Kod Parçası 8: Tasarım Sistemi Renk Değişkenleri

CSS değişkenlerinin kullanımı, tüm arayüzde tutarlı bir renk paleti sağlamakta ve olası bir tema değişikliğinin tek bir noktadan yapılabilmesine olanak tanımaktadır.

4.2. Düzen ve Ölçeklendirme

4.2.1. Genel Sayfa Düzeni

Ana sayfa düzeni CSS Flexbox ile yönetilmektedir. `body` ögesi yatay bir esnek kapsayıcı olarak tanımlanmış; EICAS ekranı ve

kontrol panelleri yan yana yerleştirilmiştir:

```
1 body {
2   background-color: #0f1318;
3   display: flex;
4   justify-content: center;
5   align-items: stretch;
6   gap: 12px;
7   margin: 0;
8   padding: 12px;
9   height: 100vh;
10  box-sizing: border-box;
11  overflow: hidden;
12 }
```

Kod Parçası 9: Ana Sayfa Düzeni

4.2.2. EICAS Ekranının Kare Oranı

EICAS göstergesinin her zaman kare biçimde görünmesi için `aspect-ratio` özelliği kullanılmaktadır [3]:

```
1 .eicas-display {
2   height: 100%;
3   aspect-ratio: 1 / 1; /* Her zaman kare */
4   background-color: #000;
5   border: 2px solid #333;
6   flex-shrink: 0;
7 }
8
9 .eicas-display svg {
10  width: 100%;
11  height: 100%;
12 }
```

Kod Parçası 10: EICAS Ekranı Oranı

4.2.3. Zoom ile Ölçeklendirme

Kontrol panellerinin yüksek piksel yoğunluklu ekranlarda okunabilir kalması için zoom özelliği kullanılmaktadır:

```
1 .control-panel {
2   zoom: 1.4; /* Standart büyütölmüş hal */
3 }
4
5 /* Firefox ve zoom desteklemeyen tarayıcılar
6   için alternatif */
7 @supports not (zoom: 1) {
8   .control-panel {
9     zoom: unset;
10    transform: scale(1.4);
11    transform-origin: top left;
12  }
```

Kod Parçası 11: Kontrol Paneli Ölçeklendirmesi

4.3. EICAS Uyarı Animasyonları

Low Oil Pressure (LOP) arızası tetiklendiğinde, ilgili metin ve arka planın yanıp sönmesi için CSS @keyframes animasyonları aşağıdaki kod bloğunda tanımlanmıştır:

```
1 @keyframes eicas-blink {
2   0%, 49% { opacity: 1; }
3   50%, 100% { opacity: 0; }
4 }
5
6 /* Siyah flas (text için) */
7 .eicas-blink-black,
8 .eicas-blink-black * {
9   fill: #000000 !important;
10  animation: eicas-blink 0.5s infinite;
11 }
12
13 /* Amber flas (kutu için) */
14 .eicas-blink-amber,
15 .eicas-blink-amber * {
16   fill: #ffcd59 !important;
17   animation: eicas-blink 0.5s infinite;
18 }
19
20 /* Sabit amber (10 saniye sonra) */
21 .eicas-steady-amber,
22 .eicas-steady-amber * {
23   fill: #ffcd59 !important;
24 }
```

Kod Parçası 12: EICAS Yanıp Sönme Animasyonu

4.4. Senaryo Aktif Parlama Efekt

Bir uçuş senaryosu oynatılırken kontrol paneli etrafında mavi bir parlama (glow) efekti oluşturulmaktadır. Bu efekt CSS box-shadow ve @keyframes ile gerçekleştirilmiştir:

```
1 .scenario-group.scenario-active {
2   border-color: rgba(0, 229, 255, 0.35);
3   animation: scenario-pulse 2.5s ease-in-out
4     infinite;
5 }
6
7 @keyframes scenario-pulse {
8   0%, 100% {
9     box-shadow:
10       0 0 10px rgba(0,229,255,0.2),
11       inset 0 0 8px rgba(0,229,255,0.05);
12   }
13   50% {
14     box-shadow:
15       0 0 16px rgba(0,229,255,0.35),
```

```
15       inset 0 0 12px rgba(0,229,255,0.08)
16     ;
17   }
```

Kod Parçası 13: Senaryo Aktif Parlama Animasyonu

4.5. Birim Dönüşüm Aracının CSS Animasyonu

Birim dönüşüm aracındaki kayan vurgulayıcı animasyonu CSS transition özelliği kullanılarak gerçekleştirilmiştir. Kaydırıcı, geçiş sırasında konumunu yumuşak bir şekilde değiştirmektedir:

```
1 .unit-toggle-slider {
2   position: absolute;
3   top: 1px; bottom: 1px;
4   width: calc(50% - 1px);
5   background-color: var(--accent);
6   border-radius: 2px;
7   left: 1px;
8   transition: left 0.28s cubic-bezier(0.4, 0,
9     0.2, 1);
10 }
11
12 /* Sağ tarafa geçiş */
13 .unit-toggle-slider.right {
14   left: calc(50%);
```

Kod Parçası 14: Toggle Widget Kayan Vurgulayıcı

4.6. Duyarlı Tasarım

Dar ekranlar ve mobil cihazlar için medya sorgusu uygulanmıştır:

```
1 @media screen and (max-width: 900px) {
2   body {
3     overflow: auto;
4     height: auto;
5     min-height: 100vh;
6     align-items: flex-start;
7   }
8   .control-panel {
9     zoom: 1 !important;
10    max-height: none;
11    overflow-y: visible;
12  }
13 }
```

Kod Parçası 15: Mobil için Medya Sorgusu

5. JavaScript ile Motor Fiziği Simülasyonu

5.1. Mimari Genel Bakış

script.js dosyası sekiz ana bölüme ayrılmaktadır:

1. Sabitler ve sistem değişkenleri
2. Durum değişkenleri
3. Başlangıç temizliği ve yardımcı fonksiyonlar
4. Motor metrikleri ve limit kontrolleri
5. Arıza/uyarı tetikleyicileri
6. Kullanıcı arayüzü etkileşimleri
7. Yakıt tüketimi ve motor döngüsü
8. Otomatik senaryo oynatımı

5.2. Sabitler ve Durum Yönetimi

5.2.1. Rotasyon Sabitleri

İbrelerin açısal hareketi iki temel sabit üzerine kuruludur:

```
1 const ROTATION_START_ANGLE = 0; // İbrenin
   baslangic acisi
2 const ROTATION_END_ANGLE   = 211.5; // İbrenin
   maksimum acisi
3 const PERCENTAGE_DIVISOR    = ROTATION_END_ANGLE
   / 2; // Yuzdeye bolme carpani
```

Kod Parçası 16: Rotasyon Sabitleri

PERCENTAGE_DIVISOR değeri şu eşitlikten türetilmektedir:

$$\text{PercentageDivisor} = \frac{\text{RotationEndAngle}}{2} = \frac{211.5}{2} = 105.75 \quad (1)$$

5.2.2. Eylemsizlik (Inertia) Sabiti

Simülasyonun fiziği, tüm değer geçişlerinin anlık değil, doğal bir atalet eğrisiyle gerçekleşmesini sağlamak üzerinedir. Temel eylemsizlik sabiti şöyle tanımlanmaktadır:

```
1 const BASE_INERTIA = 10 * Math.pow(10, -3); //
   0.010
2 let currentInertiaFactor = BASE_INERTIA;
```

Kod Parçası 17: Eylemsizlik Sabiti

5.2.3. Durum Değişkenleri

Uygulama, motor ve sistem durumunu çok sayıda durum değişkeniyle takip etmektedir [6]:

```
1 // --- Yakıt ---
2 let fuelLevelLeftWing   = 8500; // LB cinsinden
   sol kanat tanki
3 let fuelLevelRightWing  = 8500; // LB cinsinden
   sag kanat tanki
4 let fuelLevelCenter     = 24000; // LB
   cinsinden merkez tank
5
6 // --- Motor Metrikleri ---
7 let currentEgtLeft      = 100; // Baslangic EGT
   degeri (C)
8 let currentN2Left       = 0; // N2 hizi (%)
```

```
9 let currentOilTempLeft  = 40; // Yag
   sicakligi (C)
10 let currentOilPressLeft = 0; // Yag basinci (
   PSI)
11 let currentOilQtyLeft   = 18.0; // Yag miktari
   (quart)
```

Kod Parçası 18: Temel Durum Değişkenleri

5.3. Motor Fizik Modeli

5.3.1. Ana Motor Güncelleme Fonksiyonu

updateEngineState(throttleValue, side) fonksiyonu, verilen gaz kolu pozisyonuna ve kanat tarafı verisine göre motor parametrelerini hesaplar ve SVG'yi, dolayısıyla gösterge ekranını günceller:

```
1 function updateEngineState(throttleValue, side)
2 {
3   const throttle = Math.min(parseFloat(
4     throttleValue), 105);
5   // Anlık hesaplananlar (atalet uygulanmaz)
6   const n1 = throttle;
7   const ff = throttle * 14.2; // Yakıt Akisi
8   // (PPH)
9   const vib = throttle * 0.015; // Titreşim
10
11   // Hedef degerler (atalet ile hedefe
12   // ulasilacak)
13   let targetN2 = throttle > 0 ? (
14     throttle * 0.8) + 20 : 0;
15   let targetEgt = 100 + (throttle * 5.5);
16   let targetOilPress = throttle > 0 ? 30 + (
17     throttle * 0.6) : 0;
18   let targetOilTemp = 40 + (throttle * 0.8);
19   ...
20 }
```

Kod Parçası 19: Motor Parametresi Hedef Hesaplamaları

5.3.2. Eylemsizlik (Atalet) Simülasyonu

Gerçek motorlarda parametreler anlık değişmez; bir ivmelenme eğrisi izler. Bu, birinci dereceden üstel yakınsama ile modellenmiştir:

```
1 // Farkli parametreler farkli hizda tepki verir
2 currentN2Left += (targetN2 -
3   currentN2Left) * currentInertiaFactor;
4 currentOilTempLeft += (targetOilTemp -
5   currentOilTempLeft) * (
6   currentInertiaFactor * 0.2);
7 currentOilPressLeft += (targetOilPress -
8   currentOilPressLeft) * (
9   currentInertiaFactor * 2.0);
10 currentEgtLeft += (targetEgt -
11   currentEgtLeft) * (currentInertiaFactor *
12   0.3);
13 currentOilQtyLeft += (targetOilQtyLeft -
14   currentOilQtyLeft) * (currentInertiaFactor
15   * 0.1);
```

Kod Parçası 20: Eylemsizlik Uygulaması

Bu formülün matematiksel ifadesi:

$$\text{CurrentVal}_{t+1} = \text{CurrentVal}_t + (\text{TargetVal} - \text{CurrentVal}_t) \times \alpha \quad (2)$$

[4]

Burada α eylemsizlik faktörüdür. Farklı parametreler için α çarpanı şöyle belirlenir:

Tablo 4: Parametre Başına Eylemsizlik Çarpanları

Parametre	Çarpan	Tepki Hızı
N2	$\alpha \times 1.0$	Normal
EGT	$\alpha \times 0.3$	Yavaş
OIL PRESS	$\alpha \times 2.0$	Hızlı
OIL TEMP	$\alpha \times 0.2$	Çok yavaş
OIL QTY	$\alpha \times 0.1$	En yavaş

5.4. İbre Açısı Hesaplamaları

5.4.1. N1 İbre Açısı

N1 göstergesi için ibre açısı, throttle değeriyle doğru orantılıdır:

```
1 updateRotationAngle(
2     'n1_needle_1',
3     (throttle / PERCENTAGE_DIVISOR) *
4     ROTATION_END_ANGLE,
5     'n1_left_sweep'
6 );
```

Kod Parçası 21: N1 İbre Açısı Güncelleme

$$\theta_{N1} = \frac{\text{Throttle}}{105.75} \times 211.5 \quad (3)$$

5.4.2. EGT İbre Açısı

EGT göstergesi 100°C'den başladığı için bir ofset uygulanmaktadır:

```
1 updateRotationAngle(
2     'egt_left_needle',
3     ((currentEgtLeft - 100) / 5.5 /
4     PERCENTAGE_DIVISOR) * ROTATION_END_ANGLE,
5     'egt_left_sweep'
6 );
```

Kod Parçası 22: EGT İbre Açısı Güncelleme

$$\theta_{EGT} = \frac{\text{EGT}_{\text{current}} - 100}{5.5 \times \text{PERCENTAGE_DIVISOR}} \times \text{RotationEndAngle} \quad (4)$$

5.4.3. N1 Referans (Bug) Açısı

Referans göstergesi belirli bir aralıkta hareket eder:

$$R = \begin{cases} 2N & \text{eğer } N \leq 105 \\ 210 & \text{eğer } N > 105 \end{cases} \quad (5)$$

5.4.4. SVG Süpürme Yayı (Sweep Arc) Hesabı

İbre kadranının taradığı yay alanı, SVG'nin stroke-dashoffset özelliğiyle oluşturulmaktadır. Fonksiyon içindeki hesap şöyle çalışır:

```
1 const circumference = 19.922; // 2 * Math.PI *
2   3.1706
3 const offset = circumference - (circumference *
4   (angle / 360));
5 sweepEl.style.strokeDashoffset = offset;
```

Kod Parçası 23: Sweep Arc Dashoffset Hesabı

$$\text{Offset} = \text{Circumference} - \left(\text{Circumference} \times \frac{\theta}{360} \right) \quad (6)$$

5.5. Limit Kontrolleri ve Renk Uyarıları

checkEngineLimits() fonksiyonu, her motor metriğini belirlenen alt ve üst sınırlarla karşılaştırarak ilgili SVG metin ve ibre öğelerine Amber veya Red CSS sınıfı uygulamaktadır:

```
1 function applyColorClass(textId, needleId,
2   value, limitAmberTop, limitRedTop,
3   limitAmberBot, limitRedBot)
```

Kod Parçası 24: Motor Limit Kontrol Fonksiyonu

Tablo 5: Motor Parametre Sınır Değerleri

Param.	Amber Min	Kırmızı Min	Amber Max	Kırmızı Max
N1 (%)	—	—	—	104.0
N2 (%)	—	—	—	104.5
EGT (°C)	—	—	640	665
OIL P.(PSI)	13	0	—	—
OIL T.(°C)	—	—	140	155

5.6. Yakıt Simülasyonu

5.6.1. Tank Yönetim Mantığı

Yakıt tüketimi, her saniye tetiklenen consumeFuel() fonksiyonu tarafından yönetilir. Sistem önce merkez tankı tüketir; merkez tank bittiğinde kanat tanklarından tüketim başlar:

```
1 function consumeFuel() {
2   const consumeRateLeft = (fuelFlowRateLeft
3     / 3600)
4     * (
5     FUEL_UPDATE_INTERVAL_MS / 100)
6     * fuelBurnMultiplier
7   ;
8   const consumeRateRight = (fuelFlowRateRight
9     / 3600)
10     * (
11     FUEL_UPDATE_INTERVAL_MS / 100)
12     * fuelBurnMultiplier
13   ;
14   if (fuelLevelCenter > 0) {
15     // Ünce merkez tanktan tüketen
16     const takenFromCenter = Math.min(
17       fuelLevelCenter, consumeRateLeft +
18       consumeRateRight);
```



```

11     fuelLevelCenter -= takenFromCenter;
12     // Merkez yetmezse kanat tanklarından
    tamamen
13     ...
14 } else {
15     // Merkez bittiyse kanat tanklarından
    tüketen
16     fuelLevelLeftWing = Math.max(0,
    fuelLevelLeftWing - consumeRateLeft);
17     fuelLevelRightWing = Math.max(0,
    fuelLevelRightWing - consumeRateRight);
18 }
19 }

```

Kod Parçası 25: Yakıt Tüketim Mantığı

Anlık tüketim hızı formülü:

$$\text{Rate} = \frac{\text{FF}}{3600} \times \frac{\text{Interval}_{ms}}{100} \times \text{BurnMult} \quad (7)$$

5.6.2. Yakıt Göstergesi Yay Hesabı

Dairesel yakıt barları, gerçek tank seviyesinin maksimum kapasite oranıyla çizilmektedir:

```

1 const circumference = 2 * Math.PI * 5.4;
2 const sweepAngle = 261;
3 const visibleCircumference = circumference * (
    sweepAngle / 360);
4 const offsetWingLeft = circumference
5 - (visibleCircumference * (
    fuelLevelLeftWing / MAX_FUEL_WING));

```

Kod Parçası 26: Yakıt Yayı Offset Hesabı

$$\text{Offset} = \text{Circum} - \left(\text{VisCircum} \times \frac{\text{FuelLevel}}{\text{MaxFuel}} \right) \quad (8)$$

5.7. Arıza ve Uyarı Sistemi

5.7.1. EICAS Uyarı Mekanizması

triggerEICASWarning() fonksiyonu, EICAS ekranındaki yanıp sönen veya sabit yanan uyarı kutularını/metinlerini kontrol eder:

```

1 function triggerEICASWarning(elementId,
    activate) {
2     const el = document.getElementById(
        elementId);
3     if (!el) return;
4
5     // Metin elementiyse farklı, kutu ise farklı
    renk sınıfı seçilir
6     const isText = elementId.includes('text');
7     const blinkClass = isText ? 'eicas-blink-
    black' : 'eicas-blink-amber';
8     const steadyClass = isText ? 'eicas-steady-
    black' : 'eicas-steady-amber';
9
10    if (activate) {
11        if (el.classList.contains(blinkClass)
        || el.classList.contains(steadyClass))
            return;
12
13        el.classList.add(blinkClass);
14        el.style.display = 'block';
15
16        // 10 saniye sonunda flaşlama durur,
        uyarı sabit (steady) duruma geçer
17        warningTimers[elementId] = setTimeout
        ((() => {

```

```

18         if (el.classList.contains(
            blinkClass)) {
19             el.classList.remove(blinkClass)
20             ;
21             el.classList.add(steadyClass);
22         }, 10000);
23     } else {
24         clearTimeout(warningTimers[elementId]);
25         el.classList.remove(blinkClass,
            steadyClass);
26         el.style.display = 'none';
27     }
28 }

```

Kod Parçası 27: EICAS Uyarı Tetikleyici

5.7.2. Arıza Türleri ve Etkileri

Tablo 6: Arıza Türleri ve Etkileri

Arıza	Motor Etkisi	Görsel Uyarı
ENG FAIL	N1/N2/EGT sıfırlanır	EGT kadranında amber <i>ENG FAIL</i> yazısı
SVO	N2 +4.5, EGT +25	Amber <i>START VALVE OPEN</i>
OFB	OIL TEMP +35, OIL PRESS -8	Amber <i>OIL FILTER BYPASS</i>
LOP	OIL PRESS ≤ 12 PSI	10 sn yanıp söner, ardından sabit amber
THRUST	Slider girişi kilitlenir	Amber <i>THRUST</i> uyarısı
LOW FUEL	Tank seviyesi 1500 LB'ye düşer	Amber <i>LOW FUEL</i> uyarısı
Crossbleed	EGT +15	<i>CROSSBLEED</i> sistem uyarısı
Low Oil Qty	OIL QTY yavaşça 3.5'e düşer	<i>LOW QTY</i> sistem uyarısı

5.7.3. Yağ Kaybı Fizik Etkisi

Düşük yağ seviyesi, hem basıncı hem sıcaklığı etkiler. Bu etki bir kayıp oranı çarpanıyla modellenmektedir:

```

1 const oilLossRatio = Math.max(0, (18.0 - qty) /
    18.0);
2 targetOilTemp += (45 * oilLossRatio); //
    Sıcaklık artar
3 targetOilPress -= (10 * oilLossRatio); //
    Basıncı düşer

```

Kod Parçası 28: Yağ Kaybı Etki Hesabı

$$\text{Ratio}_{\text{loss}} = \max\left(0, \frac{18.0 - \text{Qty}}{18.0}\right) \quad (9)$$

$$\text{Target}_{\text{OilTemp}} = \text{Target}_{\text{OilTemp}} + (45 \times \text{Ratio}_{\text{loss}}) \quad (10)$$

$$\text{Target}_{\text{OilPress}} = \text{Target}_{\text{OilPress}} - (10 \times \text{Ratio}_{\text{loss}}) \quad (11)$$

5.8. Ana Motor Döngüsü (requestAnimationFrame)

Animasyon akışkanlığını sağlamak için requestAnimationFrame() kullanılmaktadır. Bu yaklaşım, setInterval'in aksine tarayıcının yenileme hızına senkronize çalışır ve sekme arka planda olduğunda kendiliğinden duraklar [7]:

```
1 function engineLoop() {
2   // Yakıt yoksa veya motor arızalıysa itme g
3   // ücü (throttle) yaratılmaz
4   const isEngineLeftActive = ((
5     fuelLevelLeftWing > 0 || fuelLevelCenter >
6     0) && !isEngineFailLeft);
7   const isEngineRightActive = ((
8     fuelLevelRightWing > 0 || fuelLevelCenter
9     > 0) && !isEngineFailRight);
10
11   const effectiveTargetLeft =
12     isEngineLeftActive ?
13     currentThrottleTargetLeft : 0;
14   const effectiveTargetRight =
15     isEngineRightActive ?
16     currentThrottleTargetRight : 0;
17
18   ...
19
20   updateEngineState(currentThrottleLeft, 'L')
21   ;
22   updateEngineState(currentThrottleRight, 'R')
23   ;
24
25   manageSystemAlerts();
26
27   requestAnimationFrame(engineLoop);
28 }
```

Kod Parçası 29: Ana Animasyon Döngüsü

5.9. Uçuş Senaryoları

Senaryo sistemi, önceden kodlanmış zaman çizelgeleri (time-line) kullanarak motor parametrelerini, arıza durumlarını ve uçuş modlarını otomatik olarak sıralı biçimde değiştirir.

5.9.1. Zamanlayıcı Altyapısı

```
1 let scenarioTimeoutIds = [];
2
3 const schedule = (delaySec, action) => {
4   scenarioTimeoutIds.push(setTimeout(action,
5     delaySec * 1000));
6 }
```

Kod Parçası 30: Senaryo Zamanlayıcı Yardımcı Fonksiyonu

Tüm setTimeout kimlikleri scenarioTimeoutIds dizisinde saklanır; bu sayede Durdur düğmesine basıldığında tüm bekleyen olaylar clearTimeout ile iptal edilebilir.

5.9.2. Normal Kalkış Senaryosu

```
1 if (type === 'normal_takeoff') {
2   // T=0: Roelanti (20%)
3   setThrottleCommand('L', 20);
4   setThrottleCommand('R', 20);
5
6   // T=3s: Stabilizasyon (50%)
7   schedule(3, () => {
8     setThrottleCommand('L', 50);
9     setThrottleCommand('R', 50);
10   });
11   // T=8s: Kalkış Gucu + T0 Modu
12   schedule(8, () => {
```

```
13     setFlightMode('mode_to');
14     setThrottleCommand('L',
15       n1ReferenceValue);
16     setThrottleCommand('R',
17       n1ReferenceValue);
18   });
19   // T=40s: Tırmanış Gucu + CLB Modu
20   schedule(40, () => {
21     setFlightMode('mode_clb');
22     setThrottleCommand('L', 88);
23     setThrottleCommand('R', 88);
24     clearScenario();
25   });
26 }
```

Kod Parçası 31: Normal Kalkış Zaman Çizelgesi

5.9.3. Motor Arızası (V1 Cut) Senaryosu

```
1 else if (type === 'v1_cut') {
2   // T=8s: Tam kalkış gucu
3   schedule(8, () => {
4     setFlightMode('mode_to');
5     setThrottleCommand('L',
6       n1ReferenceValue);
7     setThrottleCommand('R',
8       n1ReferenceValue);
9   });
10   // T=25s: Kritik hızda (V1) sağ motor arıza
11   schedule(25, () => {
12     if (!isEngineFailRight) toggleFailure('
13     eng', 'R');
14   });
15   // T=40s: Tek motor ile CON modu
16   schedule(40, () => {
17     setFlightMode('mode_con');
18     setThrottleCommand('L',
19       n1ReferenceValue);
20     clearScenario();
21   });
22 }
```

Kod Parçası 32: V1 Cut Zaman Çizelgesi

6. Matematiksel Formüller ve Fizik Modeli

6.1. Görselleştirme ve Açık Hesaplamaları

Yüzdelik Çarpımı:

$$\text{PercentageDivisor} = \frac{\text{RotationEndAngle}}{2} = \frac{211.5}{2} = 105.75 \quad (12)$$

İbrenin Taradığı Alan (Stroke Dashoffset) Değeri:

$$\text{Offset} = \text{Circumference} - \left(\text{Circumference} \times \frac{\theta}{360} \right) \quad (13)$$

Yakıt Barları Offset Değeri:

$$\text{Offset} = \text{Circum} - \left(\text{VisCircum} \times \frac{\text{FuelLevel}}{\text{MaxFuel}} \right) \quad (14)$$

N1 İbre Açısı:

$$\theta_{N1} = \left(\frac{\text{Throttle}}{\text{PercentageDivisor}} \right) \times \text{RotationEndAngle} \quad (15)$$

EGT İbre Açısı:

$$\theta_{EGT} = \left(\frac{\frac{EGT_{current} - 100}{5.5}}{\text{PercentageDivisor}} \right) \times \text{RotationEndAngle} \quad (16)$$

N1 Referans (Bug) Açısı:

$$\theta_{ref} = \theta_{start} + (\text{Percentage} \times (\theta_{end} - \theta_{start})) \quad (17)$$

6.2. Temel Motor Parametreleri

$$N_1 = \text{Throttle} \quad (18)$$

$$FF = \text{Throttle} \times 14.2 \quad (19)$$

$$VIB = \text{Throttle} \times 0.015 \quad (20)$$

$$TAT = -12 + (\text{Throttle} \times 0.1) \quad (21)$$

$$\text{Target}_{N2} = (\text{Throttle} \times 0.8) + 20 \quad (\text{Thr} > 0) \quad (22)$$

$$\text{Target}_{EGT} = 100 + (\text{Throttle} \times 5.5) \quad (23)$$

$$\text{Target}_{OilP.} = 30 + (\text{Throttle} \times 0.6) \quad (\text{Thr} > 0) \quad (24)$$

$$\text{Target}_{OilT.} = 40 + (\text{Throttle} \times 0.8) \quad (25)$$

6.3. Eylemsizlik (Inertia) Simülasyonu**Temel Eylemsizlik Sabiti:**

$$\text{BaseInertia} = 10 \times 10^{-3} = 0.01 \quad (26)$$

Genel Eylemsizlik Yumuşatma Formülü:

$$\text{CurrentVal}_{t+1} = \text{CurrentVal}_t + (\text{TgtVal} - \text{CurrentVal}_t) \times \text{Inertia} \quad (27)$$

Parametre başına eylemsizlik katsayıları:

$$N2_{current} += (\text{Target}_{N2} - N2_{current}) \times \alpha \quad (28)$$

$$\text{Temp}_{current} += (\text{Target}_{Temp} - \text{Temp}_{current}) \times (\alpha \times 0.2) \quad (29)$$

$$\text{Press}_{current} += (\text{Target}_{Press} - \text{Press}_{current}) \times (\alpha \times 2.0) \quad (30)$$

$$\text{EGT}_{current} += (\text{Target}_{EGT} - \text{EGT}_{current}) \times (\alpha \times 0.3) \quad (31)$$

$$\text{Qty}_{current} += (\text{Target}_{Qty} - \text{Qty}_{current}) \times (\alpha \times 0.1) \quad (32)$$

6.4. Arıza ve Sistem Etkileşimleri**Yağ Kayıp Oranı:**

$$\text{Ratio}_{loss} = \max \left(0, \frac{18.0 - \text{Qty}}{18.0} \right) \quad (33)$$

Yağ Kaybının Sıcaklığa Etkisi:

$$\text{Target}_{OilTemp} = \text{Target}_{OilTemp} + (45 \times \text{Ratio}_{loss}) \quad (34)$$

Yağ Kaybının Basınca Etkisi:

$$\text{Target}_{OilPress} = \text{Target}_{OilPress} - (10 \times \text{Ratio}_{loss}) \quad (35)$$

6.5. Tüketim ve Birim Çevrimleri**Anlık Yakıt Tüketim Hızı:**

$$\text{Rate} = \left(\frac{FF}{3600} \right) \times \left(\frac{\text{Interval}_{ms}}{100} \right) \times \text{BurnMult} \quad (36)$$

Celsius'tan Fahrenheit'a Çevrim:

$$F = \text{round} \left(C \times \frac{9}{5} \right) + 32 \quad (37)$$

LB'dan KG'ya Dönüşüm:

$$\text{Fuel}_{KG} = \text{Fuel}_{LB} \times 0.453592 \quad (38)$$

7. Kullanılan Araçlar ve Geliştirme Ortamı**7.1. Kullanılan Diller ve Araçlar**

Bu proje; herhangi bir derleme aracı, paket yöneticisi veya sunucu tarafı yazılımı gerektirmeyen, doğrudan tarayıcı üzerinde çalışan bir web uygulamasıdır.

Tablo 7: Kullanılan Araçlar

Araç / Dil	Kullanım Amacı
HTML5	Sayfa yapısı ve doğrudan (inline) SVG kullanımı
CSS3	Görsel tasarım, düzen ve temel animasyonlar
JavaScript (ES6+)	Gösterge hareketleri, olay yönetimi ve senaryo akışı
SVG 1.1	Ekranda yer alan vektörel çizimler
OCR-B Pro	EICAS ekranındaki yazı tipi
Inkscape 1.4.2	SVG çizimlerinin tasarımı ve düzenlenmesi

7.2. JavaScript Kullanımı

Projenin kodlanmasında modern JavaScript (ES6+) standartlarından faydalanılmıştır. Öne çıkan kullanımlar şunlardır:

- `const / let` - Değişken tanımlamaları
- `requestAnimationFrame` - Tarayıcıyla uyumlu, akıcı animasyon döngüsü sağlamak için
- `setTimeout / clearTimeout` - Senaryolardaki zamanlamaların yönetimi
- `setInterval` - Yakıt tüketiminin saniyelik hesaplanması
- `Math.min, Math.max, Math.abs` - İbre ve değerlerin sınırlarını belirlemek için
- `classList` metotları - CSS sınıflarını dinamik olarak değiştirip görselliği yönetmek için
- `style.strokeDashoffset` - SVG üzerindeki dairesel barların animasyonu için

7.3. SVG Kullanımı

Arayüzdeki göstergeler için SVG'nin sağladığı şu özellikler kullanılmıştır:

- Ekran boyutu değiştiğinde bozulmayı önlemek için `viewBox` niteliği
- Dairesel ilerleme çubuklarının hareketi için `stroke-dasharray` ve `stroke-dashoffset` [5]
- İbreleri döndürmek için `transform: rotate()`
- Her parçanın kendi merkezi etrafında dönmesi için `transform-box: fill-box`
- Ekranda yanıp sönen veya gizlenmesi gereken öğeler için `display` özelliği

7.4. Tarayıcı Uyumluluğu

Uygulama aşağıdaki tarayıcılarda test edilmiştir:

- **Google Chrome** - Sorunsuz çalışmaktadır.
- **Mozilla Firefox** - `zoom` özelliği yerine `transform: scale()` kullanılarak uyumluluk sağlanmıştır.
- **Microsoft Edge** - Chrome ile aynı altyapıyı kullandığı için benzer şekilde sorunsuz çalışmaktadır.

7.5. Yayınlama ve Kurulum

Proje, GitHub Pages üzerinden ücretsiz olarak yayınlanmaktadır. Bilgisayarda yerel olarak çalıştırmak için ise ekstra bir kurulum gerektirilmemektedir, sadece `index.html` dosyasının bir tarayıcıda açılması yeterli olmaktadır.

8. AI Kullanımı

Projenin geliştirilmesinde farklı yapay zeka ajanlarından faydalanılmıştır. Her bir ajanın güçlü ve zayıf yönleri göz önünde bulundurularak ilgili konularda buna göre destek alınmıştır. Örneğin araştırma, bilgi toplama ve kodun geliştirilme aşamasında Gemini 3.1 Pro'dan; arayüz tasarımı, bitmiş kodun gözden geçirilmesi, optimizasyon önerileri ve hata ayıklama süreçlerinde

ise Claude Sonnet 4.6'dan destek alınmıştır. Bu yaklaşım, her iki modelin de güçlü yönlerinden faydalanarak daha verimli bir geliştirme süreci sağlamıştır.

8.1. EICAS Ekranının Çizimi ve SVG İşlemleri

Kullanılacak platformun seçimi ve izlenecek yolun belirlenmesi konuları grubun üye sayısına, üyelerin kodlama hakkındaki bilgi birikimine ve geçmiş deneyimlere dayalı olmasına karşın, bu platformda ve belirlenen yolda projeyi gerçekleştirmek için gerekli teknik detayların öğrenilmesi ve uygulanması aşamasında yapay zeka ajanlarından destek alınmıştır.

Ekranın çizim aşamasında; Inkscape uygulamasının kullanımı hakkında yardım almak, çizimin HTML koduna entegre edilebilmesi için ID atama işleminin detayları, JavaScript ile bu parçaların nasıl kontrol edilebileceği, arayüz öğelerinin ekrana nasıl yerleştirilmesi gerektiği gibi konularda Gemini 3.1 Pro'dan destek alınmıştır.

8.2. Kodlama

HTML ve JavaScript kodlama süreçlerinde Gemini 3.1 Pro modelinin desteğine başvurulmuştur. Yapay zekaya prompt vermeden önce arayüzün nasıl olması gerektiği, neye benzeyeceği, arayüz öğelerinin işlevleri ve kullanılacak algoritmalar ile matematiksel formüller etrafında tartışılmış ve planlanmış, ardından bu plan doğrultusunda adım adım kodlama gerçekleştirilmiştir. Tüm kodun tek parça halinde oluşturulması gibi bir yol izlenmemiştir.

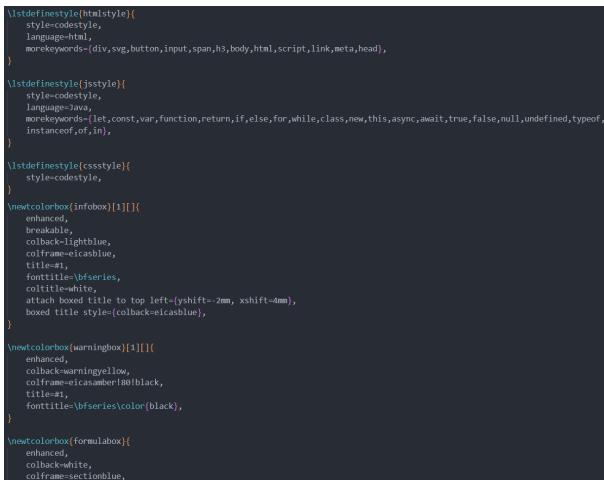
Örneğin önce N1 ve EGT ibresinin hareketi için gerekli kodlama yapılmış, ardından bu kod geliştirilerek ibrenin taradığı alanın farklı renkte olması sağlanmış, sırayla diğer ekran öğeleri statik olarak eklenmiş ve ilerleyen süreçlerde de her bir statik öğeyi dinamik hale getirecek fonksiyonlar geliştirilmiştir. Kodun her bir parçası oluşturulurken, o parçanın ne işe yaradığı, nasıl çalıştığı ve diğer parçalarla nasıl etkileşime gireceği gibi konulara odaklanılmıştır. Bu sayede kodun her bir bölümü, üzerinde detaylıca düşünülmüş ve optimize edilmiş bir şekilde geliştirilmiştir.



Son olarak ise Claude Sonnet 4.6 modelinin yardımı ile CSS kodu yazılmıştır. Bu aşamada arayüz simetrisinde, renk paletinde ve panellerin yerleşiminde pek çok defa değişiklik yapılmış, farklı düzenler denenmiş ve en uygun yerleşim sağlanmıştır. Ayrıca CSS değişkenleri ve geçiş efektleri kullanılarak modern bir görünüm elde edilmiştir. Aşağıdaki görsellerde projenin geliştirilme sürecinde farklı aşamalarda oluşturulan arayüz tasarımlarından bazıları yer almaktadır:



8.3. Raporlama



Raporun LaTeX formatında yazılması tercih edilmiştir. Bu nedenle LaTeX kurulumu, çalışma ortamının hazırlanması, sayfa

düzeninin oluşturulması, resimlerin, tabloların, formüllerin, kod bloklarının ve referansların nasıl eklenebileceği gibi bilgiler yapay zeka ve LaTeX dökümantasyonu yardımı ile öğrenilmiştir.

Kod blokları, formül kutucukları ve bilgi kutucukları gibi yapıların stilleri Claude Sonnet 4.6 modeline yaptırılmıştır. Rapor metninin yazımında ise bazı teknik kavramların açıklanması, tek bir paragraf halinde yazılan metinlerin içeriğinin değiştirilmeden daha anlaşılır listeler haline getirilmesi ve bitmiş rapordaki gözden kaçan yazım hataları ile devrik cümlelerin tespit edilmesi için Gemini 3.1 Pro'dan yardım alınmıştır. Bunun dışında rapor metninin tamamı el ile yazılmıştır.

9. Sonuç ve Değerlendirme

9.1. Genel Değerlendirme

Bu projede, Boeing 737 serisi uçakların EICAS ekranının gerçeğe yakın bir web simülasyonu geliştirilmiştir. Arayüzün FCOM belgeleri referans alınarak sıfırdan hazırlanmış olması projenin gerçekçiliğini artırmaktadır. İbrelerin anında değil eylemsizlikle hareket etmesi, farklı motor parametrelerinin farklı hızlarda tepki vermesi ve yakıt sisteminin çalışması gibi detaylar sistemin doğasına uygun şekilde modellenmiştir. Arka planda bir sunucuya ihtiyaç duymadan sadece web dilleriyle çalıştığı için uygulama son derece taşınabilir ve kolay erişilebilir durumdadır. Kullanıcılar, normal uçuş prosedürlerini deneyimleyebilir, motor arızalarını tetikleyebilir ve EICAS uyarı sisteminin nasıl çalıştığını gözlemleyebilirler.

9.2. Proje Sürecindeki Kazanımlar

Bu projenin geliştirilmesi aşamasında pratik yapma imkânı bulunan konular şunlardır:

- Inkscape ile detaylı vektörel (SVG) çizimler oluşturma ve bu dosyaların kod yapısını anlama
- HTML içine gömülü SVG elemanlarını JavaScript ile anlık olarak kontrol etme
- Temel matematiksel formüllerle fiziksel hareketleri koda dökme
- CSS değişkenleri ve geçiş efektleriyle modern arayüz tasarımı uygulama
- Belirli senaryoları sırayla çalıştıran zamanlayıcı sistemleri kurma

Kaynaklar

- [1] Boeing Commercial Airplanes, “Engines, APU: Over/Under Displays”, *737-600/700 Flight Crew Operations Manual*, 2003
- [2] Mozilla Developer Network (MDN), “Using CSS custom properties (variables)”, *MDN Web Docs*. Erişim: https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties.
- [3] W3C, “CSS Conditional Rules Module Level 3”, W3C Candidate Recommendation. Erişim: <https://www.w3.org/TR/css3-conditional/>.

- [4] “Exponential smoothing,” *Wikipedia, The Free Encyclopedia*, Wikimedia Foundation. Eriřim: https://en.wikipedia.org/wiki/Exponential_smoothing
- [5] Mozilla Developer Network (MDN), “stroke-dashoffset”, *MDN Web Docs*. Eriřim: <https://developer.mozilla.org/en-US/docs/Web/SVG/Attribute/stroke-dashoffset>.
- [6] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.
- [7] Mozilla Developer Network (MDN), “window.requestAnimationFrame()”, *MDN Web Docs*. Eriřim: <https://developer.mozilla.org/en-US/docs/Web/API/window/requestAnimationFrame>.